

CS1632: Requirements

Wonsun Ahn

What are requirements?

- Specifications of software that define expected behavior
 - Often collected into an SRS, *Software Requirements Specification*
 - The SRS comes in legal binders hundreds of pages long
 - And, yes, the SRS is often legally binding (no pun intended)
- *Requirements engineering*: Managing and documenting requirements
- Requirements drives → Testing drives → Development
 - Requirements engineering is where software quality assurance starts

Requirements Evolve

- Requirements are not set in stone and do evolve
 - The testing infrastructure and code implementation must also evolve too
 - Managing requirements is crucial to keep everyone on the same page
- Bad requirements engineering can cause *requirements creep*

Requirements Creep

www.blogcmmi.com



What the customer said



What was understood



What was planned



What was developed



What was described by the business analyst



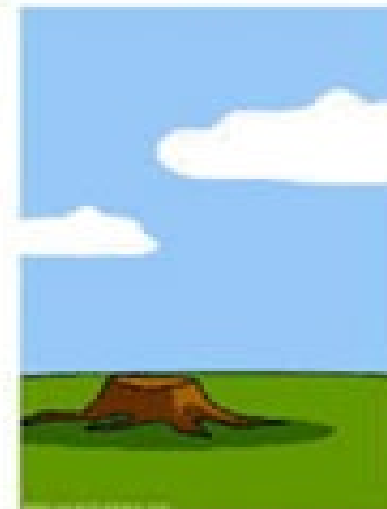
What was documented



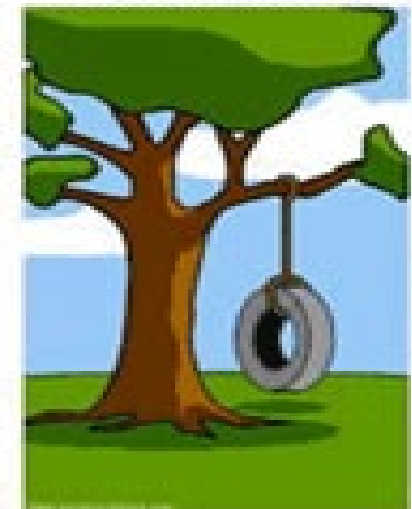
What was deployed



The customer paid for...



How was the support



What the customer really needed

Requirements Validation

Requirements Validation prevents Requirements Creep

- *Requirements Verification*: Are we building the **software right**?
 - This verifies the software against the requirements (e.g. through testing)
- *Requirements Validation*: Are we building the **right software**?
 - This validates the requirements against needs.
 1. Validates requirements by interviewing stakeholders to see that they:
 - Match actual stakeholder needs (Validity check)
 - Are technically feasible (Realism check)
 - Are testable (Verifiability check)
 2. Validates requirements are unambiguous / consistent / complete

Case Study: Requirements Validation

- Slightly modified requirements for the president program:
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print “can be US president”.
 - If the named person is not mature enough to be president, the system shall print “cannot be US president”.
- Are these good requirements?

Validity Check

- Requirements must align with stakeholders needs and wants
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print “can be US president”.
→ Do stakeholders only need the president to be mentally fit?
 - If the named person is not mature enough to be president, the system shall print “cannot be US president”.

Validity Check

- **Misconception:** Stakeholders know what they want
Truth: What looks good on paper is often a flop when seen in real life
 - **Misconception:** “More is better” – more features, more customization
Truth: “Less is more” – usability suffers with “more”
 - **Misconception:** Stakeholders are limited to end-users
Truth: Stakeholders also include operators, managers, investors, ...
- ☛ Do early prototyping and demos in front of stakeholders

Realism Check

- It must be realistic to **implement** the requirements
 - Must be realistic in terms technology / budget / timeline
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print "can be US president".
→ Do we have the technical capacity to determine mental fitness?
 - If the named person is not mature enough to be president, the system shall print "cannot be US president".

Verifiability Check

- It must be feasible to **test** the requirements
 - Must be objectively verifiable within budget and timeline
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print "can be US president".
→ Is there an agreed upon way to verify mental fitness?
 - If the named person is not mature enough to be president, the system shall print "cannot be US president".

Ambiguity Check

- Requirements should not be open to interpretation
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print “can be US president”.
 - If the named person is not mature enough to be president, the system shall print “cannot be US president”.
→ What do you mean by mature enough? How enough is enough?

Consistency Check

- Requirements must be internally consistent
 - Requirements must not contradict each other.
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
 - If the named person is mentally fit to be president, the system shall print "can be US president".
 - If the named person is not mature enough to be president, the system shall print "cannot be US president".
 - What if the person is mentally fit but immature?
(In other words, there is an overlap in the two equivalence classes.)

Completeness Check

- Requirements should cover all aspects of a system
 - If you care that something should occur a certain way, it should be specified
- What's wrong with these requirements?
 - A name of a person shall be given as a commandline argument
→ What if argument is not a name of a person? What if no arguments given?
 - If the named person is mentally fit to be president, the system shall print "can be US president".
 - If the named person is not mature enough to be president, the system shall print "cannot be US president".
→ What if person is mentally unfit but mature? Can or cannot be president?

Pitfall: Specifying HOW, not WHAT

1. Specifying the “how” may not achieve the “what” user has in mind
 - Users care about what the software does, not how it happens
2. Specifying how restricts developers from improving implementation
3. Specifying how often fails the verifiability test
 - Often source code is not available to end user to verify the “how”
 - Even if available, end users typically lack technical capability to verify

Pitfall: Specifying HOW, not WHAT

- **BAD**: The system shall have dual modular redundancy for all modules.
- **GOOD**: The system shall have a mean-time-between-failures of 100 years.

- **BAD**: The system shall be comprised of 100 CPUs.
- **GOOD**: The system shall support up to 100 concurrent users with a response time of less than 10 ms.

Functional Requirements and Non-functional Requirements

Functional and Non-Functional Requirements

- **Functional Requirements**

- Specify functions the system is expected to perform (an “output”).
- The system shall **do** X on input Y.

- **Non-Functional Requirements (Quality Attributes)**

- Specify qualities system should exhibit, not a functionality.
- The system shall **be** X during operation.

Quality Attribute Categories

- Reliability
- Usability
- Accessibility
- Performance
- Safety
- Supportability
- Security

You can see why quality attributes are sometimes called “-ility” requirements!

Functional Requirement Examples

- **Req 1:** System shall **return** "NONE" if no elements match the query.
- **Req 2:** System shall **throw an exception** on illegal parameters.
- **Req 3:** System shall **turn on** HIPRESSURE light at 100 PSI.
- **Req 4:** HIPRESSURE light shall **be** red.
 - Note verb is “be”, but it still describes functional behavior

Quality Attribute Examples

- **Req 1:** The system shall **be** protected against unauthorized access.
- **Req 2:** The system shall **be** easily extensible and maintainable.
- **Req 3:** The system shall **be** portable to other processor architectures.
- **Req 4:** The system shall **have** 99% uptime.
 - Note verb is not “be”, but it still describes a quality (reliability)

Quality Attributes are Difficult to Test

- Why? Because they are literally qualitative.
- Can be subjective. (e.g., How reliable is reliable?)
- Often difficult to measure. (e.g., How do you measure reliability?)

Solution

*Agree with stakeholders upon **quantifiable requirements** that ensure quality.*

Converting Qualitative to Quantitative

- **Performance:** response time, transactions per second
- **Reliability:** Mean-time-between-failures (MTBF)
- **Robustness:** How many simultaneous failures can system cope with
- **Portability:** Number of systems targeted, or how long it takes to port
- **Usability:** Average amount of time required for training

Qualitative to Quantitative Example

- Quality attributes should be expressed in a quantitative way
 - Or else they are ambiguous
- Example
 - **BAD:** The system shall be highly usable.
 - **GOOD:** Over 90% of users shall be able to operate the software after one hour of training.
- Example
 - **BAD:** The system shall be reliable enough to be used in a space station.
 - **GOOD:** The system shall have a mean-time-between-failures of 100 years.

Now Please Read Textbook Chapters 5

- (Optional) If you are interested in further reading:

ISO/IEC/IEEE International Standard - Systems and software engineering -- Life cycle processes -- Requirements engineering (ISO/IEC/IEEE 29148)

<https://ieeexplore-ieee-org.pitt.idm.oclc.org/document/8559686>